第7章 第8节 MCP 通信协议

- 来源: https://class.imooc.com/lesson/5281#mid=104937
- 阶段: 阶段三: 【工具与协议】让AI连接世界
- 周: 第7周 MCP：模型上下文协议
- 模块: MCP通信协议与实战
- 课节: 图文 1-1 MCP 通信协议

1-1 MCP 通信协议

第7章 第8节 MCP 通信协议

> [!TIP] 阅读指南 前面学习了 MCP 的三大核心能力：Resources、Tools 和 Prompts。但有个问题一直没有回答：AI 应用和 MCP Server 之间到底是怎么通信的？ 有同学可能会想："直接调用函数不就行了吗？比如 server.get_resource(uri)。" 如果你还停留在这种思想，那可能还是没有理解 MCP 的核心价值。MCP 本身就是要把工具和应用分离——如果 AI 应用直接调用 Server 的函数，那岂不是又把两者耦合在一起了？那和传统的 Function Calling 有什么区别？ 实际上，MCP Server 更像是一个 HTTP 服务（比如 RESTful API），HTTP服务通常都运行在另外一个进程，甚至根本都不在你的本地计算机上(可能在遥远的服务器上)。 但它和 HTTP 服务又有很大的不同：HTTP 用的是 GET/POST 请求，MCP 用的是什么？ HTTP 跑在 80/443 端口，MCP呢？ HTTP服务器通常借助各类Web 服务器，比如TomCat等，那MCP Server 怎么启动？ HTTP 有 URL 路由，MCP 怎么找到对应的资源或工具？ 这一节深入 MCP 的底层，看看它的通信协议和架构设计。

8.1 MCP 与 HTTP 的异同

相同之处：都是 Client-Server 架构

MCP 和 HTTP 都采用了 Client-Server 架构：

Client-Server 架构

HTTP 的世界：

```
浏览器（Client）          Web 服务器（Server）
     |
     ├─ GET /api/users ──────→
     | ←────── 返回 JSON ──────┤
     |
     ├─ POST /api/login ─────→
     | ←────── 返回 token ─────┤
```

MCP 的世界：

```
AI 应用（Client）          MCP Server
     |
     ├─ resources/list ──────→
     | ←────── 返回笔记列表 ──────┤
     |
     ├─ tools/call ──────────→
     | ←────── 创建笔记成功 ──────┤
```

共同特点：

- Client 发起请求，Server 处理并返回结果
- Server 是独立运行的，与 Client 解耦合
- 支持多个 Client 连接同一个 Server

解耦意味着什么

「Server 与 Client 解耦合」不仅仅是为了架构整洁——它是 MCP 安全模型的根基。

还记得第 3 节 Resources 示例里那句「AI 应用不能直接读本地文件」吗？原因就在这里：如果 AI 应用（Client）能直接读写磁盘，那就意味着一个 LLM 驱动的程序拥有对整个文件系统的完全访问权。prompt 注入、模型判断失误——任何一个环节出问题，都可能导致文件泄露或被篡改。

MCP 的做法很明确：AI 应用不碰磁盘。它的角色是编排调度——通过协议向 Server 发请求（“给我 authentication.md 的内容”），拿到结果后转发给 LLM，LLM 生成回答后展示给用户。真正打开文件、读取内容、写入改动的物理 I/O 操作，全部发生在 MCP Server 的进程内——Server 在自己的身体里执行 fs.readFile()，而不是让 AI 应用去碰硬盘。Client 能访问什么、不能访问什么，完全由 Server 在启动配置中声明——在 mcpServers 里白纸黑字写清楚。

用一个简单的类比来记住这个原则：Server 就像一个图书管理员，AI 是读者。读者不能自己进书库翻找，必须告诉管理员要哪本书，管理员去取来。书库的钥匙只在管理员手里。

核心差异：通信方式不同

虽然都是 Client-Server，但实现方式完全不同：

特性	HTTP	MCP
协议格式	HTTP/1.1, HTTP/2	JSON-RPC 2.0
请求方法	GET, POST, PUT, DELETE	统一的 method 字段
路由方式	URL 路径（/api/users/123）	方法名（resources/read）
传输层	TCP 端口（80/443）	stdio / HTTP / WebSocket
状态码	200, 404, 500 等	JSON-RPC 错误码（-32xxx）
身份认证	Cookie, JWT, OAuth	由启动参数控制

关键区别解读：

HTTP 用 URL，MCP 用方法名

```
HTTP 的路由方式：
GET /api/notes/123      → 获取笔记
POST /api/notes         → 创建笔记
DELETE /api/notes/123   → 删除笔记

MCP 的方法调用方式：
{"method": "resources/read", "params": {"uri": "file://notes/123"}}
{"method": "tools/call", "params": {"name": "create_note"}}
{"method": "tools/call", "params": {"name": "delete_note"}}
```

HTTP 通过 URL + 请求方法 来区分操作，MCP 通过 方法名 来表达意图。

HTTP 跑在端口上，MCP 更灵活

MCP 最常用的通信方式是 stdio（标准输入输出）。与需要监听端口、配置路由的 HTTP 不同，stdio 直接通过进程的标准流进行数据交换。这种「进程内即通信」的设计，让本地工具免去了端口冲突和网络握手的麻烦，安全性也更高。

当然，对于需要跨网络协作的场景，MCP 也完全支持 HTTP+SSE 或 WebSocket。其核心逻辑是：本地工具优先用 stdio，跨网络服务则用网络协议。

HTTP 用状态码，MCP 用错误对象

```
HTTP 的错误返回：
HTTP/1.1 404 Not Found
{"error": "Note not found"}

MCP 的错误返回：
{
  "jsonrpc": "2.0",
  "id": 1,
  "error": {
    "code": -32601,
    "message": "Method not found"
  }
}
```

HTTP 用 HTTP 状态码（200/404/500），MCP 用 JSON-RPC 错误码（-32xxx）。

为什么 MCP 不直接用 HTTP

既然 HTTP 这么成熟，为什么还要折腾这一套？核心原因有三点：

- 极简的本地集成：stdio 无需网络协议栈，对本地文件系统和工具的访问更自然。

极简的本地集成

- RPC 逻辑更契合：AI 调用工具本质上是「执行动作」（RPC），而非「访问资源」（REST）。
- 传输层解耦：MCP 不绑定在特定传输层，可以根据性能和场景在 stdio、WebSocket 之间无缝切换。

8.2 JSON-RPC 2.0 协议详解

我们已经看到 MCP 在架构设计上更倾向于「执行动作」而非「访问资源」，那么它必然需要一套标准化的「对话语言」。正如人类交流需要语法，AI 应用与 MCP Server 之间的对话则完全建立在 JSON-RPC 2.0 协议之上。这一节剥开 MCP 的外壳，直击这套支撑起万千工具调用的底层通信逻辑。

什么是 JSON-RPC

JSON-RPC 是一种轻量级的远程过程调用协议，用 JSON 格式传输数据。

轻量级的远程过程调用协议

> 技术的回响 如果你是一位资深开发者，看到这里可能会感到亲切。这种模式在本质上非常接近早年流行的 RPC（Remote Procedure Call）技术。它回归了「指令式」的交互直觉：直接告诉对方我要执行哪个函数，而不是像 REST 那样去操作资源。

类比理解：

```
传统函数调用（同一进程内）：
result = calculator.add(5, 3)

远程过程调用（跨进程）：
发送请求：{"method": "add", "params": [5, 3]}
接收响应：{"result": 8}
```

JSON-RPC 让跨进程调用函数像本地调用一样简单。

三种消息类型

JSON-RPC 2.0 定义了三种消息：

请求（Request）

客户端调用服务端的方法：

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/call",
  "params": {
    "name": "create_note",
    "arguments": {
      "title": "MCP学习笔记",
      "content": "今天学了JSON-RPC协议"
    }
  }
}
```

字段说明：

- jsonrpc: 协议版本，通常设置为 "2.0"

- id: 请求的唯一标识。服务端返回响应时必须携带相同的 ID，客户端以此来区分这个回复对应的是哪一次请求。
- method: 要调用的方法名
- params: 方法参数（可选）

响应（Response）

服务端返回的结果：

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "content": [
      {
        "type": "text",
        "text": "笔记创建成功！文件路径：notes/MCP学习笔记.md"
      }
    ]
  }
}
```

- id: 必须与对应请求中的 id 完全一致，否则客户端会认错人。
- result: 成功时的返回值
- error: 失败时的错误信息（二选一）

错误响应示例：

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "error": {
    "code": -32602,
    "message": "Invalid params",
    "data": "参数 'title' 不能为空"
  }
}
```

通知（Notification）

单向消息，不需要响应：

```
{
  "jsonrpc": "2.0",
  "method": "notifications/resources/updated",
  "params": {
    "uri": "file://notes/new_note.md"
  }
}
```

关键特征： 没有 id 字段，接收方不需要回复。

关键特征：

使用场景：

- Server 通知 Client：资源列表更新了
- Client 通知 Server：用户取消了操作

标准错误码

MCP 遵循 JSON-RPC 2.0 的标准错误码：

错误码	含义	说明
-32700	Parse error	JSON 解析失败
-32600	Invalid Request	请求格式不正确
-32601	Method not found	方法不存在
-32602	Invalid params	参数无效
-32603	Internal error	服务器内部错误
-32000 到 -32099	Server error	自定义服务器错误

> 错误码使用规则： 标准错误码（-32768 到 -32000）： 由 JSON-RPC 协议预定义，不能修改 用于协议层错误（JSON 解析失败、方法不存在等） MCP Server 必须遵循这些标准 自定义错误码（-32000 到 -32099）： 可以自己定义 用于业务逻辑错误（文件不存在、权限不足等） 示例： -32000: 文件不存在 -32001: 权限被拒绝 -32002: 资源已锁定 **建议：**在 MCP Server 中定义自己的错误码常量，保持一致性

实际例子：

```
{
  "jsonrpc": "2.0",
  "id": 2,
  "error": {
    "code": -32601,
    "message": "Method not found",
    "data": "方法 'delete_all_notes' 不存在，可用方法：create_note, search_notes,
add_thoughts"
  }
}
```

8.3 下一节预告

了解了 JSON-RPC 的消息格式后，下一个问题是：这些 JSON 消息是怎么从 Client 传到 Server 的？MCP 提供了三种传输层选择——stdio、SSE 和 WebSocket，各有适用场景。下一节将详细讲解它们的工作原理和选择依据，还会介绍 MCP Server 从启动到关闭的完整生命周期。

8.4 ■ 学点英语

中文	English	音标	说明
JSON-RPC 2.0	JSON-RPC 2.0	/ˈdʒeɪsɑːn ɑːr piː siː tuː pɔɪnt oʊ/	基于JSON的轻量级远程过程调用协议，MCP的基础通信协议
请求-响应模式	Request-Response Pattern	/rɪˈkwest rɪˈspɑːns ˈpætərn/	Client发送带id的请求，Server返回同id响应的通信模式
单向通知	One-Way Notification	/wʌn weɪ ˌnoʊtɪfɪˈkeɪʃn/	不带id的消息，发送方不期待响应的单方向通信
方法命名空间	Method Namespace	/ˈmeθəd ˈneɪmspeɪs/	用点号分层的命名方式如resources/list、tools/call